

Лабораторная работа 4. Сегментация изображений

В предыдущих лабораторных работах вы решали задачу **классификации** — модель отвечала на вопрос «что изображено на картинке?», присваивая изображению одну метку класса. В этой работе вы перейдёте к задаче **семантической сегментации** — модель будет отвечать на вопрос «где именно на картинке находится каждый объект?», присваивая метку класса **каждому пикселю** изображения. В этой работе вы:

- Познакомитесь с архитектурой **encoder-decoder** со skip-connections — ключевой архитектурной идеей для задач, где нужно не только «понять» изображение, но и сформировать выход того же пространственного разрешения.
- Освоите специфичные для сегментации **метрики качества** — IoU и Dice Score.
- Используйте **предобученный encoder**, применив transfer learning к новому типу задачи.
- Проанализируете, на каких областях изображения модель ошибается и почему.

Выбор датасета

Для сегментации нужен датасет с **попиксельными масками** — изображениями, в которых каждый пиксель размечен номером класса.

Обязательные требования

- **Задача:** семантическая сегментация (не менее 2 классов, включая фон).
- **Размер изображений:** не более 256×256 пикселей (при необходимости уменьшить).
- **Объём выборки:** не менее 500 изображений (после фильтрации/предобработки)

Примеры подходящих датасетов

Датасеты разделены по уровню сложности. Сложность определяется числом классов, разнообразием сцен и типичным качеством результатов (mIoU), достижимым простой моделью U-Net. Время обучения указано ориентировочно для модели U-Net с ~2M параметров, изображения 128×128, batch_size=16, 30 эпох.

Простой уровень (бинарная сегментация, 2 класса)

Рекомендуется для первого знакомства с сегментацией. Объекты обычно крупные, границы чёткие, достижимый mIoU > 0.85.

Датасет	Объём	Сложность (1 - 5)	Время CPU	Время GPU	Источник
Carvana Car Masking	5 088	1	~2–3 ч	~15–25 мин	Kaggle
LGG Brain MRI Segmentation	3 929	1	~1.5–2.5 ч	~10–20 мин	Kaggle
Breast Ultrasound (BUSI)	780	1	~20–40 мин	~3–8 мин	Kaggle

Датасет	Объём	Сложность (1 - 5)	Время CPU	Время GPU	Источник
Data Science Bowl 2018 (Nuclei)	670	2	~15–30 мин	~3–6 мин	Kaggle
ISIC 2018 Skin Lesion	2 594	2	~1–2 ч	~8–15 мин	ISIC Archive

Средний уровень (3–20 классов)

Несколько классов, более разнообразные сцены. Требуется более глубокая модель и аугментация. Достижимый mIoU: 0.50–0.80.

Датасет	Объём	Сложность (1 - 5)	Время CPU	Время GPU	Источник
Oxford-IIIT Pet	7 349	2	~3–5 ч	~20–40 мин	Kaggle
CamVid	701	3	~15–30 мин	~2–5 мин	Kaggle
Pascal VOC 2012	2 913	3	~1–2 ч	~8–15 мин	Kaggle
Semantic Drone Dataset	400	3	~10–20 мин	~2–4 мин	Kaggle
Cityscapes (resized)	5 000	4	~2–4 ч	~15–30 мин	cityscapes-dataset.com

Сложный уровень (20+ классов, крупные датасеты)

Много классов, сложные многообъектные сцены, сильный дисбаланс классов. Рекомендуется только при наличии GPU. Достижимый mIoU: 0.25–0.50.

Датасет	Объём	Сложность (1 - 5)	Время CPU	Время GPU	Источник
ADE20K	20 210	4	~8–15 ч	~40–80 мин	MIT
COCO-Stuff 10K	10 000	4	~4–8 ч	~25–50 мин	GitHub
BDD100K Segmentation	10 000	4	~4–8 ч	~25–50 мин	bdd-data.berkeley.edu
Mapillary Vistas	25 000	5	Не рекомендуется	~1.5–3 ч	mapillary.com
COCO Panoptic	118 000	5	Не рекомендуется	~4–8 ч	cocodataset.org

Примечание: время обучения указано для ориентира и может отличаться в 2–3 раза в зависимости от конкретного оборудования, размера модели и размера изображений. При увеличении разрешения с 128×128 до 256×256 время обучения возрастает примерно в 3–4 раза.

Рекомендации по обучению

Использование GPU для обучения

Обучение нейронных сетей на CPU, как вы уже могли заметить, может занимать **часы**, тогда как на GPU те же модели обучаются за **минуты**. Для выполнения лабораторных работ GPU не

обязателен (можно выбрать простой датасет), но существенно ускоряет эксперименты и позволяет работать с более сложными задачами.

Бесплатные GPU-платформы

Платформа	GPU	VRAM	Ограничения	Рекомендация
Google Colab	NVIDIA T4	15 GB	Сессия до ~4–12 ч, GPU может не быть доступен в пиковые часы	Лучший вариант для большинства задач
Kaggle Notebooks	NVIDIA P100 или T4	16 GB	30 часов GPU в неделю, сессия до 12 ч	Отличная альтернатива Colab, многие датасеты доступны прямо на платформе

Настройка Google Colab (рекомендуемый вариант)

1. **Подключите GPU:** откройте ноутбук в Colab → **Среда выполнения** → **Сменить среду выполнения** → выберите **T4 GPU**.
2. **Проверьте, что GPU доступен:**

```
import torch
print(f"CUDA доступна:{torch.cuda.is_available()}")
print(f"GPU:{torch.cuda.get_device_name(0)}")
print(f"VRAM:{torch.cuda.get_device_properties(0).total_mem/ 1024**3:.1f} GB")
```

3. **Перенесите модель и данные на GPU:**

```
device= torch.device('cuda' if torch.cuda.is_available()else 'cpu')
print(f"Используется:{device}")

model= UNet(in_channels=3,num_classes=num_classes).to(device)
```

4. **В обучающем цикле** не забывайте переносить батчи на GPU:

```
for x_batch, y_batch in train_loader:
    x_batch= x_batch.to(device)
    y_batch= y_batch.to(device)

    logits= model(x_batch)
    loss= criterion(logits, y_batch)
    # ...
```

5. **При оценке** переносите данные обратно на CPU для визуализации:

```
pred_mask= logits.argmax(dim=1).cpu().numpy()
```

6. **Сохраняйте модель** на Google Drive, чтобы не потерять результаты при отключении сессии:

```
from google.colabimport drive
drive.mount('/content/drive')

# Сохранение
torch.save(model.state_dict(), '/content/drive/MyDrive/unet_best.pth')

# Загрузка
model.load_state_dict(torch.load('/content/drive/MyDrive/unet_best.pt
h'))
```

Настройка Kaggle Notebooks

1. **Создайте ноутбук:** kaggle.com → **Code** → **New Notebook**.
2. **Подключите GPU:** в правой панели **Settings** → **Accelerator** → **GPU T4 ×2** или **GPU P100**.
3. **Загрузка датасета:** если датасет уже есть на Kaggle, подключите его через **Add Data** в правой панели — он будет доступен в **/kaggle/input/**.
4. **Сохранение результатов:** используйте **/kaggle/working/** для выходных файлов. После завершения сессии их можно скачать.
5. **Контроль лимита:** проверяйте оставшееся GPU-время в **Settings** → **Quota**.

Универсальный код для работы на CPU и GPU

Чтобы ваш ноутбук корректно работал и на CPU, и на GPU без изменений, используйте следующий паттерн:

```
device= torch.device('cuda' if torch.cuda.is_available()else 'cpu')
print(f"Устройство:{device}")

model= model.to(device)

# В обучающем цикле:
for x_batch, y_batchin train_loader:
    x_batch= x_batch.to(device)
    y_batch= y_batch.to(device)
    # ... обучение ...

# При визуализации:
image_np= x_batch[0].cpu().permute(1,2,0).numpy()
mask_np= pred_mask[0].cpu().numpy()
```

Советы по экономии GPU-времени

Совет	Эффект
Уменьшите разрешение (128×128 вместо 256×256) при прототипировании	Ускорение в ~4 раза
Уменьшите batch_size , если не хватает VRAM (ошибка CUDA out of memory)	Позволяет обучить более крупную модель
Отлаживайте на CPU с маленьким датасетом, на GPU — только финальное обучение	Экономия GPU-квоты
Сохраняйте чекпоинты каждые N эпох	Не потеряете прогресс при отключении сессии
Используйте num_workers > 0 в DataLoader	Ускоряет загрузку данных (не всегда работает в Colab)

Что делать, если нет доступа к GPU

Если ни одна из бесплатных платформ недоступна:

1. **Выберите простой датасет** из простой категории (BUSI, Nuclei — менее 1000 изображений).
2. **Уменьшите разрешение** до 64×64 или 128×128.
3. **Используйте лёгкую модель** — U-Net с 3 уровнями и начальным числом фильтров 16–32 (вместо 64).
4. **Сократите число эпох** при прототипировании (5–10 эпох для проверки, полное обучение — на ночь).
5. **Запускайте обучение на ночь** — 4–6 часов на CPU вполне достаточно для простого датасета.

Подготовка данных

Подготовка данных для сегментации отличается от классификации, так как модель обучается предсказывать класс **для каждого пикселя** изображения.

1. Каждому изображению должна соответствовать **маска того же размера**. Маска содержит **целочисленные метки классов: 0, 1, 2, ..., num_classes-1**.

Маска:

- имеет размер [H, W]
 - имеет тип **long**
 - **не нормализуется**
2. При аугментации **геометрические преобразования** (resize, flip, crop) должны применяться **одинаково** к изображению и маске. При этом используется разная интерполяция:
 - **Изображение** — билинейная интерполяция
 - **Маска** — метод ближайшего соседа

Нарушение этого правила приводит к появлению дробных или несуществующих меток классов в маске, что сделает обучение некорректным.

3. Рекомендуется использовать библиотеку `albumentations`, которая автоматически применяет одно и то же случайное преобразование и к изображению, и к маске с правильной

интерполяцией.

Установка:

```
pip install albumentations
```

Пример:

```
import albumentations as A
from albumentations.pytorch import ToTensorV2

transform_train = A.Compose([
    A.Resize(128, 128),
    A.HorizontalFlip(p=0.5),
    A.RandomBrightnessContrast(p=0.3),
    A.Normalize(mean=(0.485, 0.456, 0.406),
                 std=(0.229, 0.224, 0.225)),
    ToTensorV2(),
])

transform_val = A.Compose([
    A.Resize(128, 128),
    A.Normalize(mean=(0.485, 0.456, 0.406),
                 std=(0.229, 0.224, 0.225)),
    ToTensorV2(),
])
```

При использовании `albumentations` преобразования применяются так:

```
transformed = transform(image=image_numpy, mask=mask_numpy)
image = transformed['image'] # тензор [C, H, W], float, нормализованный
mask = transformed['mask'] # тензор [H, W], long, без нормализации
```

4. **Dataset** для сегментации должен возвращать пару (image, mask):

```
from torch.utils.data import Dataset

class SegmentationDataset(Dataset):
    def __init__(self, images, masks, transform=None):
        self.images = images
        self.masks = masks
        self.transform = transform

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        image = self.images[idx] # numpy [H, W, C], uint8
        mask = self.masks[idx] # numpy [H, W], int
```

```

if self.transform:
    transformed = self.transform(image=image, mask=mask)
    image = transformed['image']
    mask = transformed['mask'].long()

return image, mask

```

Если вы используете датасет из `torchvision` (`OxfordIIITPet`, `VOCSegmentation`), маски могут приходить в формате `PIL Image`. В этом случае необходимо преобразовать их в `numpy`-массив и убедиться, что значения классов корректны (целые числа от `0` до `num_classes-1`).

Проверка корректности данных

Перед началом обучения **обязательно** выполните проверки:

```

image, mask = dataset[0]

print(f"image shape: {image.shape}")      # ожидается: [3, 128, 128]
print(f"mask shape: {mask.shape}")        # ожидается: [128, 128]
print(f"mask dtype: {mask.dtype}")        # ожидается: torch.int64
print(f"mask classes: {mask.unique()}")   # значения от 0 до num_classes-1

```

Убедитесь, что:

- Пространственные размеры изображения и маски **совпадают**.
- Маска содержит **только допустимые классы** (целые числа от `0` до `num_classes-1`).
- В маске **нет дробных значений** (если есть — проблема с интерполяцией).
- Тип маски — `torch.int64` (`long`).

Перед обучением визуализируйте **не менее 5 примеров** в формате:

```
[Исходное изображение]    [Маска]
```

Проверьте визуально:

- Совпадают ли объекты на изображении и маске.
- Нет ли «размытых» границ классов в маске (признак неправильной интерполяции).
- Корректно ли отображаются все классы.

Метрики качества сегментации

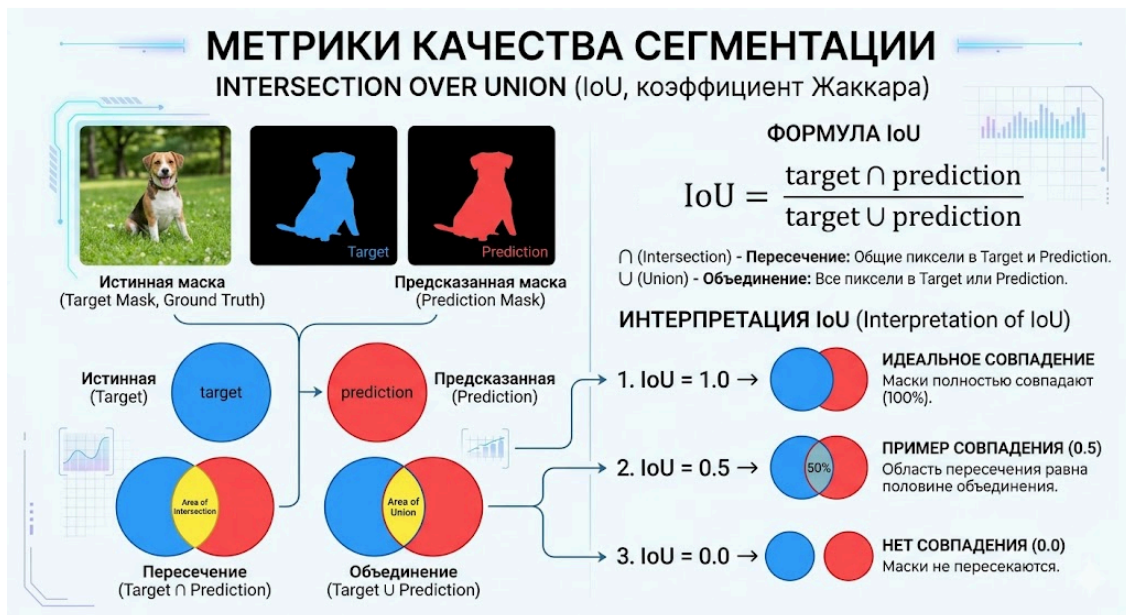
В отличие от классификации, где основная метрика — ассигасу, для сегментации используются другие метрики, учитывающие пространственное совпадение предсказанной маски с истинной.

Intersection over Union (IoU, коэффициент Жаккара)

Для каждого класса вычисляется отношение площади пересечения предсказанной и истинной областей к площади их объединения:

$$IoU = \frac{\text{target} \cap \text{prediction}}{\text{target} \cup \text{prediction}}$$

Пересечение ($A \cap B$) состоит из пикселей, найденных как в маске предсказания, так и в основной маске истины, тогда как объединение ($A \cup B$) просто состоит из всех пикселей, найденных либо в маске предсказания, либо в целевой маске.



Mean IoU (mIoU) — IoU, усреднённый по всем классам — основная метрика сегментации.

$$mIoU = \frac{1}{C} \sum_{i=1}^C \frac{|prediction_i \cap target_i|}{|prediction_i \cup target_i|}$$

Dice Score (коэффициент Сёренсена-Дайса)

$$Dice = \frac{2|target \cap prediction|}{|target| + |prediction|}$$

Dice Score тесно связан с IoU, но придаёт больший вес пересечению. Используется особенно часто в медицинской сегментации.

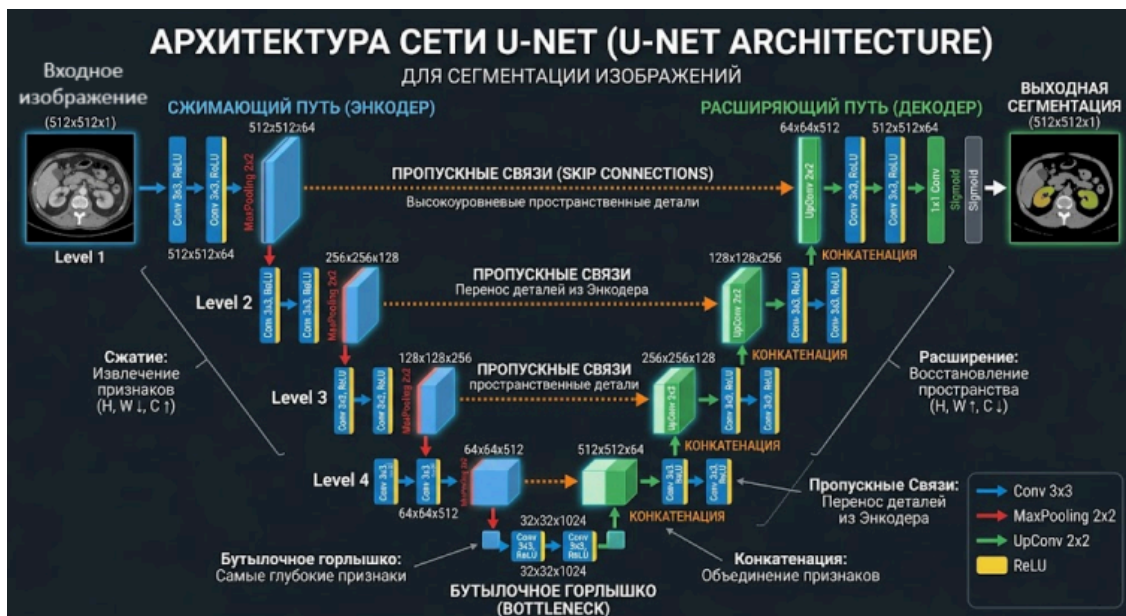
Архитектура U-Net: ключевая идея

U-Net — это архитектура типа **encoder-decoder** со **skip-connections**, которая позволяет:

- извлекать **семантические признаки** (encoder)
- восстанавливать **пространственную структуру** (decoder)

Skip-connections — это соединения, которые передают карты признаков из encoder'a напрямую в соответствующий уровень decoder'a (обычно через конкатенацию по каналам).

Без них decoder получает только сильно сжатое представление из bottleneck и теряет мелкие детали (границы объектов, текстуры). Skip-connections позволяют decoder'у использовать **и высокоуровневую семантику** (из глубоких слоёв), **и низкоуровневые детали** (из ранних слоёв).



Задание 1. Обязательное для всех (максимальная оценка — 8)

Постройте и обучите модель для семантической сегментации изображений.

Часть А. Построение U-Net

Реализуйте архитектуру U-Net. Модель должна содержать:

Encoder:

- Не менее 3 уровней свёрточных блоков с уменьшением пространственного разрешения.
- Каждый блок: свёртки + BatchNorm + ReLU + пулинг (или свёртка с `stride=2`).
- Число фильтров увеличивается с глубиной (например, $64 \rightarrow 128 \rightarrow 256$).

Bottleneck:

- Свёрточный блок в самом «дне» сети (максимальное сжатие).

Decoder:

- Не менее 3 уровней с увеличением пространственного разрешения.
- Каждый уровень: `ConvTranspose2d` (или `Upsample` + `Conv2d`) + свёртки + BatchNorm + ReLU.
- Число фильтров уменьшается (зеркально encoder'у).

Skip-connections:

- На каждом уровне карты признаков encoder'а конкатенируются с картами признаков decoder'а по каналам.
- Конкатенация: `torch.cat([encoder_features, decoder_features], dim=1)`.

- После конкатенации число каналов удваивается — учтите это в следующем свёрточном слое.

Выходной слой:

- Свёртка 1×1 (`nn.Conv2d(in_channels, num_classes, kernel_size=1)`) — преобразует карты признаков в логиты для каждого класса.
- **Без softmax** (он включён в `nn.CrossEntropyLoss`).

Пример структуры:

```
import torch
import torch.nn as nn

class UNet(nn.Module):
    def __init__(self, in_channels=3, num_classes=2):
        super().__init__()

        # ===== Encoder =====
        self.enc1 = self._block(in_channels, 64)
        self.pool1 = nn.MaxPool2d(2)

        self.enc2 = self._block(64, 128)
        self.pool2 = nn.MaxPool2d(2)

        self.enc3 = self._block(128, 256)
        self.pool3 = nn.MaxPool2d(2)

        # ===== Bottleneck =====
        self.bottleneck = self._block(256, 512)

        # ===== Decoder =====
        self.up3 = nn.ConvTranspose2d(512, 256, kernel_size=2, stride=2)
        self.dec3 = self._block(256 + 256, 256) # skip-connection

        self.up2 = nn.ConvTranspose2d(256, 128, kernel_size=2, stride=2)
        self.dec2 = self._block(128 + 128, 128)

        self.up1 = nn.ConvTranspose2d(128, 64, kernel_size=2, stride=2)
        self.dec1 = self._block(64 + 64, 64)

        # ===== Output =====
        self.out_conv = nn.Conv2d(64, num_classes, kernel_size=1)

    def _block(self, in_ch, out_ch):
        """Двойная свёртка: Conv → BN → ReLU → Conv → BN → ReLU"""
        return nn.Sequential(
            nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1),
            nn.BatchNorm2d(out_ch),
```

```

        nn.ReLU(inplace=True),

        nn.Conv2d(out_ch, out_ch, kernel_size=3, padding=1),
        nn.BatchNorm2d(out_ch),
        nn.ReLU(inplace=True),
    )

def forward(self, x):
    # ===== Encoder =====
    e1 = self.enc1(x) # [B, 64, 128, 128]
    e2 = self.enc2(self.pool1(e1)) # [B, 128, 64, 64]
    e3 = self.enc3(self.pool2(e2)) # [B, 256, 32, 32]

    # ===== Bottleneck =====
    b = self.bottleneck(self.pool3(e3)) # [B, 512, 16, 16]

    # ===== Decoder =====
    d3 = self.up3(b) # [B, 256, 32, 32]
    d3 = torch.cat([d3, e3], dim=1) # skip-connection
    d3 = self.dec3(d3)

    d2 = self.up2(d3) # [B, 128, 64, 64]
    d2 = torch.cat([d2, e2], dim=1)
    d2 = self.dec2(d2)

    d1 = self.up1(d2) # [B, 64, 128, 128]
    d1 = torch.cat([d1, e1], dim=1)
    d1 = self.dec1(d1)

    return self.out_conv(d1)

```

Skip-connections объединяют признаки encoder и decoder, увеличивают число каналов (поэтому 256 + 256).

```
d3 = torch.cat([d3, e3], dim=1)
```

ConvTranspose2d увеличивает размер изображения в 2 раза, используется для восстановления разрешения.

Убедитесь, что размерности согласованы. Пропустите тестовый тензор через модель и проверьте, что выход имеет форму `[batch, num_classes, H, W]`, где H и W совпадают с входными.

```

model = UNet(in_channels=3, num_classes=num_classes)

x = torch.randn(1, 3, 128, 128)
y = model(x)

print(x.shape, "→", y.shape)

```

Ожидаемый результат:

```
[1, 3, 128, 128] → [1, num_classes, 128, 128]
```

Часть В. Обучение

1. **Функция потерь.** Используйте `nn.CrossEntropyLoss` — она работает попиксельно:

```
criterion = nn.CrossEntropyLoss()
# logits: [batch, num_classes, H, W]
# target: [batch, H, W] (long, значения 0..num_classes-1)
loss = criterion(logits, target)
```

Если ваш датасет **сильно несбалансирован** (например, фон занимает 90% пикселей), передайте веса классов:

```
class_weights= torch.tensor([0.5,2.0,...])# больший вес для редких классов
criterion= nn.CrossEntropyLoss(weight=class_weights)
```

2. **Обучите модель** с оптимизатором Adam, отслеживая:

- **Train loss** по эпохам.
- **Mean IoU на валидации** по эпохам.

Рекомендуется 20–50 эпох. Визуализируйте графики train loss и val mIoU по эпохам.

3. **Визуализируйте результаты** для не менее 6 изображений из валидационной выборки. Для каждого изображения покажите три столбца:

```
[Исходное изображение]  [Истинная маска]  [Предсказанная маска]
```

Для получения предсказанной маски:

```
model.eval()
with torch.no_grad():
    logits= model(image.unsqueeze(0))# [1, num_classes, H, W]
    pred_mask= logits.argmax(dim=1).squeeze(0)# [H, W]
```

Для визуализации масок используйте цветовую карту:

```
plt.imshow(mask, cmap='tab10', vmin=0, vmax=num_classes- 1)
```

4. **Оцените модель на тестовой выборке** — вычислите mean IoU и Dice Score.

Задание 2. Дополнительное (максимальная оценка — 10)

Улучшите модель сегментации с помощью предобученного encoder'a, проведите сравнительный анализ и исследуйте ошибки модели.

Часть А. U-Net с предобученным encoder-ом

В Лабораторной работе 3 вы использовали предобученные модели (ResNet, EfficientNet и др.) для классификации. Свёрточные слои этих моделей извлекают универсальные визуальные признаки — границы, текстуры, формы — которые полезны и для задачи сегментации. Идея: заменить encoder в U-Net на предобученный backbone и дообучить decoder.

1. **Выберите предобученную модель** (рекомендуется та же, что в Лабораторной работе 3) и используйте её как encoder.

Для ResNet18, например, encoder можно разбить на уровни так:

```
import torchvision.models as models

resnet = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)

# Уровни encoder-а (каждый уменьшает разрешение в 2 раза)
self.enc1 = nn.Sequential(resnet.conv1, resnet.bn1, resnet.relu) # /
2
self.pool1 = resnet.maxpool # /4
self.enc2 = resnet.layer1 # /4
self.enc3 = resnet.layer2 # /8
self.enc4 = resnet.layer3 # /16
```

Число выходных каналов каждого уровня можно узнать, пропустив тестовый тензор или из документации модели.

2. **Постройте decoder**, учитывая число каналов из предобученного encoder-а. Skip-connections подключаются аналогично Заданию 1.
3. **Настройте отдельные learning rate** — пониженный для предобученного encoder-а, нормальный для decoder-а (аналогично Лабораторной 3):

```
encoder_params = [...] # параметры из resnet
decoder_params = [...] # параметры нового decoder-а

opt = torch.optim.Adam([
    {'params': encoder_params, 'lr': 1e-4},
    {'params': decoder_params, 'lr': 1e-3},
])
```

4. **Обучите модель** и оцените на тестовой выборке.

Часть В. Сравнение

Сведите результаты двух моделей (U-Net с нуля vs U-Net с предобученным encoder-ом) в таблицу:

Модель	Параметры (всего)	Параметры (обуч.)	Эпох до сходимости	mIoU (val)	mIoU (test)
U-Net с нуля	...	...	...	...	...

Модель	Параметры (всего)	Параметры (обуч.)	Эпох до сходимости	mIoU (val)	mIoU (test)
U-Net + pretrained encoder	...	...	...	...	...

Дополнительно покажите **IoU по каждому классу** для обеих моделей:

Класс	IoU (с нуля)	IoU (pretrained)
Фон	...	...
Класс 1	...	...
Класс 2	...	...
...	...	...

Часть С. Анализ ошибок

1. **Визуализируйте 4–6 изображений**, на которых модель ошибается больше всего (с наименьшим IoU). Для каждого покажите:

[Исходное изображение] [Истинная маска] [Предсказанная маска] [Карта ошибок]

Карта ошибок — это бинарное изображение, где отмечены пиксели, в которых предсказание не совпадает с истиной:

```
error_map = (pred_mask != true_mask).float()
plt.imshow(error_map, cmap='Reds')
```

2. **Классифицируйте типичные ошибки.** Проанализируйте, где модель чаще всего ошибается:
 - На **границах** объектов (неточное выделение контура)?
 - На **мелких объектах** (пропускает или не замечает)?
 - Путает **похожие классы** (какие именно)?
 - Ошибается на изображениях **определённого типа** (сложный фон, необычный ракурс)?
3. **Предложите способы улучшения** (не обязательно реализовывать, достаточно обоснованных предложений):
 - Специальные функции потерь для границ (например, Boundary Loss).
 - Дополнительная аугментация для проблемных случаев.
 - Увеличение глубины сети или числа фильтров.
 - Post-processing предсказаний (CRF, морфологические операции).